一台虚拟机上的 LEBench

SOSP 2019，Ren 等人

邹浩宇　5260000157

CS340 操作系统

2026 年 11 月 3 日

论文声称什么

- 从 Linux 3.0 到 4.20，核心内核操作变慢了（图 1）。
- 例外是大块写和大块 munmap。
- 十一个原因覆盖这些跳动：安全、新功能、错误配置（表 3、图 2）。
- 关掉它们之后，Redis、Apache 和 Nginx 会变快（第 5 节）。

我们在作者的内核清单上检验第一条，并把一个安全开关当作扩展。应用程序没有跑。

实际跑的是什么

- 测量代码来自 LEBench 提交 `6aa7801`，没有改计时方法。
- 同一条 K-best 规则：五个样本落在 5% 以内，用 `clock_gettime` 计时。
- 循环缩短到 `BASE_ITER` 40，这样每次启动可以重复三次。
- Ubuntu 16.04.6 跑在 VirtualBox 里，传统 BIOS，磁盘在 `D:`。
- 每个内核一次 `grub-reboot`。panic 后回到 4.4.0-210。

为什么数字不是论文里的数字

- 客户机：2 个 vCPU，4 GB，虚拟磁盘，宿主机开了内存完整性。
- CPU：i7-13700H。Meltdown：不受影响。
- 论文的服务器是 Xeon E5-2630 v3，128 GB。
- 我们只在这台客户机上比较内核。不去对齐论文的纳秒数。

扩展：只改一个开关 `nopti`

同一内核（4.4.0-210），同一二进制，只改命令行。

- getpid：314 ns 对 295 ns。区间重叠。
- 小块读、send、recv、select、poll、epoll 的区间也重叠。
- fork：93 μs 对 254 μs。区间不重叠，方向和 KPTI 开销相反。
- 两次启动的 `/sys/.../meltdown` 都是“Not affected”。

完整的表在报告里。

`nopti` 说明了什么

- 漏洞文件写明这颗 CPU 不受 Meltdown 影响。
- getpid、读、send、recv 的变化小于各次运行之间的波动。
- 加上 `nopti` 之后 fork 更慢，这不是论文里的 KPTI 故事。
- KPTI 开销在这里没有复现出来。这颗 CPU 本来就不付这笔代价。

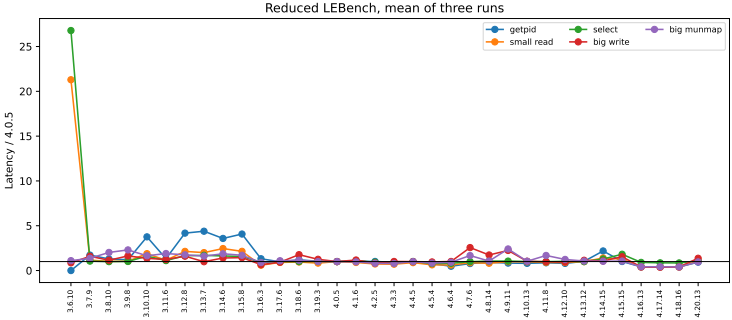
扩展：批量计时 getpid

只改一处：一次计时 1,000 次调用，再除以 1,000。K-best 和机器不变。

- 4.0.5：从 245 ± 0 ns 降到 129 ± 0 ns。区间不重叠。
- 4.4.0-210：从 314 ± 39 ns 降到 127 ± 40 ns。
- 批量之后这两个内核的区间重叠。单次计时里的变慢没有留下来。
- 4.0.5 上一次空跑的时钟大约 130 ns。原来的 getpid 一行主要就是这一对时钟。

各内核之间

比 4.0.5 更晚的 19 个内核、627 项测量里，271 项（43%）均值更高，76 项（12%）至少高 50%。117 项的区间不重叠且均值更高。



一份没有采用的重写

另一套 C 程序用 `rdtsc` 重写了测试，结果表是空的。

- 在这台虚拟机上，`rdtsc` 不是合适的时钟。
- `select` 在计时区间里做了分配。
- 上下文切换是一次平均值，不是 K-best，也没有把优先级调到 -20。
- 它没有在 Linux 上执行过。

课程这棵树调用的是 `OS_Eval`，没有把它换掉。

结论

41 个列出的内核里，34 个完成了三次运行。3.0.7–3.5.4 的 sshd 没有应答。4.19.11 没有可下载的 modules 包。

- 论断 1：在这里不成立。更晚内核里 43% 的均值高于本机 4.0.5，12% 至少高 50%。
- 19 个更晚内核里，getpid 有 9 个明显更快，2 个更慢。
- 论断 2：`nopti` 没有复现 KPTI 开销。这颗 CPU 不受影响。
- 论断 3：没有跑，所以无法判断。
- 总体：部分复现。缺的是论文自己的结论，测量框架已经跑起来了。

两种安静的失败

- 计时器选错时，抄来的基准程序仍然会打出一个数字。
- 内核包装得上，却可能永远到不了 sshd。
- 两种情况都要留在日志里：要么是一份 CSV，要么是一行启动失败。
- 没有使用上游那个把 `grub-install` 装到分区上的做法。它可能写错磁盘。